Regression Models for Chemistry Prediction

Beginner tutorial and fully worked instructor solutions

Prof. Victor S. Batista

CHEM 584: Introduction to Machine Learning and Quantum Computing

An earlier version of this assignment was initially desiged by Dr. Jessica Freeze

Scientific objective: Predict the **first ionization energy** of elements, in kJ/mol, using five tabulated properties from the supplied periodic-table dataset: atomic weight, melting point, boiling point, molar heat capacity, and thermal conductivity.

Learning sequence: Understand the data → clean units carefully → build a linear model by hand → train ordinary least squares (OLS) → compare Ridge regression → evaluate a random forest → interpret the chemistry and limits of the predictions.

This edition is **CSV-based**: the original `periodic_table.csv` from the ZIP is embedded and decoded within the notebook. It does not require Google Drive, file uploads, or internet access. All calculations are reproducible with a fixed random seed.

Student deliverables and grading (100 points)

Section	Work to submit	Points
1. Understanding chemical data	Feature/target explanations, source-data inspection, physical units	15
2. Cleaning and splitting data	Numeric parsing, missing data report, leakage explanation	20
3. Linear regression from first principles	Hand calculation, model fit, coefficients and residuals	20
4. Ridge regression and validation	Hyperparameter choice, plots, performance table	20
5. Random forest and interpretation	Comparison, diagnostic figures, outliers and limitations	25
Total	Executed `.ipynb` and short written conclusions	**100**

Instructor edition: Fully worked conceptual answers and executable solutions appear immediately after each question. Numerical scores can vary with software-version differences; the procedures and random seed are specified.

Data file setup

Download **`periodic_table.csv`** alongside this notebook. Place it in the same working directory before running the first Python cell. In Google Colab, upload the file using the left sidebar **Files → Upload**. The code explicitly reads the CSV using `pd.read_csv`; no data is stored inside the notebook.

CSV instructions: Download `periodic_table.csv` separately. In Google Colab, run the first code cell: if the CSV is absent, you will be prompted to select and upload it. In Jupyter, put the CSV in the notebook working folder. Colab runtime uploads may need to be repeated after a runtime reset.

1. What are features, targets, and a regression model? (15 points)

A **sample** is one chemical element. A **feature** is a measured or tabulated quantity used to make a prediction. The **target** is the quantity we hope to predict.

For element i, write the five features as

$$\mathbf{x}_i = (m_i, T_{m,i}, T_{b,i}, C_{p,i}, \kappa_i),$$

where m is atomic weight, T_m and T_b are melting and boiling temperatures, C_p is molar heat capacity, and κ is thermal conductivity. The target y_i is first ionization energy (kJ/mol).

If there are N elements and five features, the **feature matrix** X has shape $(N, 5)$. Each **row** represents an element; each **column** represents a type of property. A regression model approximates y_i using X_i.

Q1(a): Why is this a *regression* rather than a *classification* problem?

Worked answer: The target is a numerical quantity on a continuous scale, such as 520.2 kJ/mol for lithium. Classification would instead choose a category, such as "high" versus "low" ionization energy. The assignment asks for numerical energy estimates, so it is a regression problem.

Q1(b): Why does a feature not necessarily *cause* a change in ionization energy?

Worked answer: A feature may correlate with ionization energy through electronic structure, period, or element group without itself being a causal mechanism. For example, boiling point depends on condensed-phase bonding and interactions, while atomic ionization is a gas-phase electronic process. Statistical prediction does not establish causal relationships.

Q1(c): What are the physical units of the five features?

Worked answer: Temperatures are nominally kelvin (K); heat capacity is typically J mol^{-1} K^{-1}; thermal conductivity is W m^{-1} K^{-1}; tabulated atomic weights are conventionally relative atomic masses (numerically comparable to molar mass in g/mol); the target is kJ/mol. We retain the dataset's conventions and flag ambiguous entries.

```
import re, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

SEED = 42
np.random.seed(SEED)
# Locate the external CSV. Colab runtimes do NOT automatically receive files
# simply because they were downloaded alongside the notebook.
# If the CSV is missing, Colab will display an interactive upload prompt.
from pathlib import Path
import os

CSV_FILENAME = "periodic_table.csv"
search_locations = [
    Path.cwd() / CSV_FILENAME,
    Path("/content") / CSV_FILENAME,             # standard Colab working folder
    Path("/mnt/data") / CSV_FILENAME,            # hosted notebook environments
]
CSV_PATH = next((p for p in search_locations if p.is_file()), None)

if CSV_PATH is None:
    try:
        from google.colab import files  # available only inside Google Colab
    except ImportError as exc:
        raise FileNotFoundError(
            f"Cannot locate {CSV_FILENAME}. Place it in the notebook's working "
            "directory, or set CSV_PATH to its full absolute path. "
            f"Current directory: {Path.cwd()}"
        ) from exc
    print(f"Please select and upload {CSV_FILENAME} from your computer.")
    uploaded = files.upload()  # user-facing Colab file chooser
    if CSV_FILENAME not in uploaded:
        raise FileNotFoundError(
            f"Expected {CSV_FILENAME}, but received: {list(uploaded)}. "
            "Please upload the supplied periodic_table.csv file and rerun this cell."
        )
    # files.upload writes the data to Colab's current working folder.
    CSV_PATH = Path.cwd() / CSV_FILENAME
    if not CSV_PATH.is_file():
        CSV_PATH.write_bytes(uploaded[CSV_FILENAME])

print(f"Loading CSV from: {CSV_PATH.resolve()}")
periodic = pd.read_csv(CSV_PATH, encoding="utf-8-sig")
required_columns = ["name", "atomic_weight", "melting_point", "boiling_point",
                    "molar_heat_capacity", "thermal_conductivity", "ionization_energies"]
missing_columns = [col for col in required_columns if col not in periodic.columns]
if missing_columns:
    raise ValueError(f"CSV is missing required columns: {missing_columns}")
print(f"Source data: {periodic.shape[0]} elements and {periodic.shape[1]} columns")
display(periodic[['name','atomic_weight','melting_point','boiling_point',
                  'molar_heat_capacity','thermal_conductivity','ionization_energies']].head(8))
```

```
Please select and upload periodic_table.csv from your computer.
```
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this

2. Clean the data without silently altering its meaning (20 points)

Real chemical tables often mix numbers and explanatory text. Example entries include `"(H2) 28.836 J/(mol·K)"`, `"25.83×10−3 W/(m·K)"`, and `"1st: 1312.0 kJ/mol"`. Simply deleting all non-digits is **incorrect**: `25.83×10−3` must become 0.02583, not 2583103.

Instructor decision: Extract a *single unambiguous numerical value* from each column. Convert explicit `×10` powers; reject ranges and entries that report multiple candidate values (e.g., different allotropes, C_p and C_v), unless a clear primary quantity can be selected. Missing values will be imputed **inside the training pipeline**: the imputer learns medians from training data and applies them to the held-out test data. We do not substitute guessed physical constants.

A first ionization energy field is parsed by the `1st:` prefix. If it is missing, that element is excluded because supervised regression requires a known target.

Q2(a): Why is it dangerous to replace missing data with full-dataset averages before splitting?

Worked answer: The median or mean would depend partly on the withheld test set, leaking test-set information into training. Even though no test target is used, the fitted preprocessing stage should learn only from training examples.

Q2(b): Why should temperature and conductivity not be treated as interchangeable numeric columns?

Worked answer: They have different dimensions and physical meanings; silently copying one column into another creates physically false input data and changes the conclusions. The source notebook's conductivity-cleaning line accidentally used the heat-capacity column; this version fixes that error.

```
FEATURES = ['atomic_weight', 'melting_point', 'boiling_point',
            'molar_heat_capacity', 'thermal_conductivity']
TARGET = 'ionization_energy_1st_kj_mol'

def parse_first_ionization(value):
    if pd.isna(value):
        return np.nan
    match = re.search(r'1st:\s*([\d,]+(?:\.\d+)?)\s*kJ/mol', str(value), re.I)
    return float(match.group(1).replace(',', '')) if match else np.nan

def parse_property(value):
    """Conservative parser: one explicit number, optional ×10^exponent.
    Reject ranges, competing phase values, and mixed Cp/Cv measurements.
    """
    if pd.isna(value):
        return np.nan
    if isinstance(value, (int, float, np.number)):
        return float(value)
    s = str(value).strip().replace('−','-').replace('–','-').replace('×','x')
    if not s or 'graphite:' in s.lower() or 'diamond:' in s.lower():
        return np.nan
    if 'Cp:' in s and 'Cv:' in s:
        return np.nan
    if (re.search(r'\d\s*[-]\s*\d', re.sub(r'x\s*10\s*\^?\s*[+-]?\d+', '', s))
            or re.search(r'\d\s*to\s*\d',s,re.I)):
        return np.nan
    # Remove parenthetical molecular formulas, e.g., (H2), (O2), but NOT units.
    s = re.sub(r'^\([A-Z][a-z]?\d*\)\s*', '', s)
    m = re.match(r'^\s*([+-]?\d[\d,]*(?:\.\d+)?)\s*(?:x\s*10\s*(?:\^)?\s*([+-]?\d+))?',s)
    if not m:
        return np.nan
    value = float(m.group(1).replace(',',''))
    if m.group(2) is not None:
        value *= 10**int(m.group(2))
    return value

clean = periodic[['name'] + FEATURES + ['ionization_energies']].copy()
for col in FEATURES:
    clean[col] = clean[col].map(parse_property)
clean[TARGET] = clean['ionization_energies'].map(parse_first_ionization)
clean = clean.drop(columns='ionization_energies').dropna(subset=[TARGET]).reset_index(drop=True)
assert clean['name'].is_unique
assert clean[TARGET].notna().all()
print('Rows with valid first ionization energy:',len(clean))
print('\nMissing feature values (before imputation):')
print(clean[FEATURES].isna().sum().to_string())
display(clean.head())
```

```
# Unit checks for three representative entries.
for name in ['Hydrogen','Lithium','Neon']:
    r = clean.loc[clean.name.eq(name)]
    if len(r):
        print(name, r[['thermal_conductivity',TARGET]].iloc[0].to_dict())
```

3. Derive ordinary least-squares linear regression (20 points)

One feature, worked entirely by hand

Suppose three hypothetical measurements are $(x, y) = (1, 2), (2, 3), (3, 5)$. Fit a straight line $\hat{y} = b + wx$. The average values are $\bar{x} = 2$ and $\bar{y} = 10/3$. The least-squares slope is

$$w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{3}{2} = 1.5,$$

and the intercept is $b = \bar{y} - w\bar{x} = 10/3 - 3 = 1/3$. Thus $\hat{y} = 1/3 + 1.5x$.

The model's predictions for $x = 1, 2, 3$ are $11/6$, $10/3$, and $29/6$; the residuals $y - \hat{y}$ are $1/6$, $-1/3$, and $1/6$. Their squared sum is $1/6$, so the mean-squared error is $1/18 \approx 0.0556$.

Many features

With five predictors, the model is

$$\hat{y_i} = b + w_1 x_{i1} + w_2 x_{i2} + w_3 x_{i3} + w_4 x_{i4} + w_5 x_{i5}.$$

The intercept b allows a nonzero prediction even when all features are zero. We minimize mean-squared error:

$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y_i})^2.$$

In matrix notation, append an intercept column of ones to form $\widetilde{X} = [\mathbf{1}\ X]$. If invertible, the ordinary least-squares solution is $\widehat{\beta} = (\widetilde{X}^{\mathsf{T}} \widetilde{X})^{-1} \widetilde{X}^{\mathsf{T}} y$. Practical implementations use numerically stable solvers (QR or SVD), avoiding explicit matrix inversion.

Q3(a): What does a positive regression coefficient mean?

Worked answer: Holding other included features fixed, increasing that feature increases the *model's predicted* target, provided the model is appropriate. It is not proof of a causal effect, and correlated features may make coefficients unstable.

Q3(b): Why do we report results on a test set?

Worked answer: Training error measures fit to examples that the algorithm has already seen. A held-out set tests whether the fitted relationships generalize to other elements.

```
# Verify the arithmetic of the simple worked example.
x_toy = np.array([1.,2.,3.])
y_toy = np.array([2.,3.,5.])
w_toy = np.sum((x_toy-x_toy.mean())*(y_toy-y_toy.mean())) / np.sum((x_toy-x_toy.mean())**2)
b_toy = y_toy.mean()-w_toy*x_toy.mean()
print('Toy slope:',w_toy,'intercept:',b_toy)
print('Toy predictions:',b_toy+w_toy*x_toy)
print('Toy MSE:',np.mean((y_toy-(b_toy+w_toy*x_toy))**2))
assert np.isclose(w_toy,1.5) and np.isclose(b_toy,1/3)

X = clean[FEATURES]
y = clean[TARGET]
names = clean['name']
X_train,X_test,y_train,y_test,names_train,names_test = train_test_split(
    X,y,names,test_size=0.25,random_state=SEED)
print('Training:',X_train.shape, 'Test:',X_test.shape)

ols = Pipeline([('imputer', SimpleImputer(strategy='median')),
                ('scaler', StandardScaler()),
                ('model',LinearRegression())])
ols.fit(X_train,y_train)
print('OLS standardized coefficients:',dict(zip(FEATURES,ols.named_steps['model'].coef_.round(3))))
print('OLS intercept (training-target units):',ols.named_steps['model'].intercept_)
```

Interpreting coefficients after standardization

Because conductivity and temperature have dramatically different numerical scales, the pipeline first creates standardized training features

$$z_j = (x_j - \mu_j)/s_j,$$

where μ_j and s_j are calculated **only on the training set**. Coefficients then represent the change in the predicted ionization energy (kJ/mol) for a one-standard-deviation increase in a feature, holding the other standardized features fixed. They should not be interpreted as effects of a one-unit increase in the original property.

The default `LinearRegression()` includes an intercept; it does **not** require manually adding a column of ones. Scaling makes the learned intercept equal to the training-target mean when the imputed-scaled feature columns are centered.

4. Ridge regression: why shrink coefficients? (20 points)

Ordinary least squares can be unstable when input features are correlated. Ridge regression discourages large coefficients by solving

$$\min_{b,\mathbf{w}} \left\{ \sum_i (y_i - b - \mathbf{x}_i^{\mathsf{T}}\mathbf{w})^2 + \alpha \sum_j w_j^2 \right\}.$$

The parameter $\alpha \geq 0$ controls regularization. The intercept is **not penalized** in scikit-learn's Ridge implementation. Ridge cannot remove all limitations of a misspecified model: it only changes the tradeoff between fitting the data and keeping coefficients small.

Q4(a): What happens as α increases?

Worked answer: The slopes are pushed toward zero. This can reduce sensitivity to sampling noise but eventually leads to underfitting if the penalty is too strong.

Q4(b): Why choose α with cross-validation rather than the test set?

Worked answer: Cross-validation repeats train/validation splits within the training partition, allowing hyperparameter tuning without using the test targets. The test set is evaluated only after the choice is finalized.

```
# Ridge hyperparameter search on training data only.
ridge_pipe = Pipeline([('imputer',SimpleImputer(strategy='median')),
                       ('scaler', StandardScaler()),('model',Ridge())])
cv = KFold(n_splits=5, shuffle=True, random_state=SEED)
search = GridSearchCV(ridge_pipe, {'model__alpha':[0.01,0.1,1,10,100,1000]},
                      scoring='neg_mean_absolute_error',cv=cv,refit=True)
search.fit(X_train,y_train)
ridge = search.best_estimator_
print('Ridge alpha selected by training-only CV:',search.best_params_['model__alpha'])
print('Mean CV MAE (kJ/mol):',-search.best_score_)
plt.figure(figsize=(6,4))
params=np.array([d['model__alpha'] for d in search.cv_results_['params']])
plt.semilogx(params,-search.cv_results_['mean_test_score'],marker='o')
plt.xlabel('Ridge alpha');plt.ylabel('Cross-validated MAE (kJ/mol)')
plt.title('Training-only hyperparameter comparison');plt.grid(alpha=.3);plt.show()
```

5. Compare the regressors and evaluate individual elements (25 points)

A **random forest** averages many regression decision trees. Each tree recursively splits the data, using decision boundaries that can approximate nonlinear relationships. Averaging trees tends to reduce individual-tree variance. It does not guarantee good extrapolation beyond the observed range of properties.

For a set of n test examples and residuals $e_i = \hat{y_i} - y_i$:

$$\mathrm{MAE} = \frac{1}{n}\sum_i |e_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i e_i^2}, \qquad R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2}.$$

Both MAE and RMSE have units kJ/mol. RMSE penalizes large errors more heavily; R^2 is dimensionless and can be **negative** on a test set.

Q5(a): Why is a smaller training error not sufficient evidence that a random forest is better?

Worked answer: A flexible model may learn patterns and noise peculiar to the training examples. It is preferable only when it improves performance on genuinely held-out elements and the differences are meaningful relative to data uncertainty.

Q5(b): Why are very high or low ionization energies especially difficult to predict?

Worked answer: Extreme values can be sparsely represented. Unusual electronic configurations and group-specific trends may not be captured by the five condensed-matter properties. A prediction that interpolates near common elements does not automatically

extrapolate to chemically unusual ones.

```
rf = Pipeline([('imputer',SimpleImputer(strategy='median')),
               ('model',RandomForestRegressor(n_estimators=250,min_samples_leaf=2,
                                              random_state=SEED,n_jobs=1))])
rf.fit(X_train,y_train)
models={'OLS':ols,'Ridge':ridge,'Random forest':rf}

def metrics(ytrue,ypred):
    return {'MAE (kJ/mol)':mean_absolute_error(ytrue,ypred),
            'RMSE (kJ/mol)':np.sqrt(mean_squared_error(ytrue,ypred)),
            'R2':r2_score(ytrue,ypred)}
rows=[]
for title,model in models.items():
    for subset,Xs,ys in [('train',X_train,y_train),('test',X_test,y_test)]:
        rows.append({'Model':title,'Subset':subset,**metrics(ys,model.predict(Xs))})
score_table=pd.DataFrame(rows)
print('Numerical model comparison:')
display(score_table.round(3))

fig,axs=plt.subplots(1,3,figsize=(15,4))
for ax,(title,model) in zip(axs,models.items()):
    pred=model.predict(X_test)
    ax.scatter(y_test,pred,alpha=.75)
    lims=[min(y_test.min(),pred.min()),max(y_test.max(),pred.max())]
    ax.plot(lims,lims,'k--',linewidth=1)
    ax.set(xlabel='Reference (kJ/mol)',ylabel='Predicted (kJ/mol)',title=title)
plt.tight_layout();plt.show()
```

```
# Detailed residual table, sorted by largest error for each model.
residual_tables = {}
for title,model in models.items():
    pred=model.predict(X_test)
    tab=pd.DataFrame({'Element':names_test.to_numpy(),
                      'Reference (kJ/mol)':y_test.to_numpy(),
                      'Prediction (kJ/mol)':pred})
    tab['Residual (kJ/mol)']=tab['Prediction (kJ/mol)']-tab['Reference (kJ/mol)']
    tab['Absolute error (kJ/mol)']=tab['Residual (kJ/mol)'].abs()
    residual_tables[title]=tab.sort_values('Absolute error (kJ/mol)',ascending=False)
    print('Largest errors:',title)
    display(residual_tables[title].head(5).round(2))

plt.figure(figsize=(8,4))
for title,model in models.items():
    residual=model.predict(X_test)-y_test.to_numpy()
    plt.scatter(y_test,residual,label=title,alpha=.7)
plt.axhline(0,color='black',ls='--')
plt.xlabel('Reference ionization energy (kJ/mol)')
plt.ylabel('Residual (prediction - reference, kJ/mol)')
plt.legend();plt.title('Held-out residual diagnostics');plt.show()
```

Chemical interpretation and critical limitations (worked answers)

Q5(c): Which model is best? Use the printed **test** MAE and RMSE table to identify the model with the smallest error for this particular fixed split. The answer is dataset- and split-dependent; a small difference in metrics is not proof of universal superiority. Also inspect whether training-to-test gaps suggest overfitting.

Q5(d): Can these results establish a physics-based prediction law? No. First ionization energy primarily reflects isolated-atom electronic structure, including valence-shell configuration, effective nuclear charge, and shielding. Melting point, boiling point, and conductivity are correlated periodic properties but are not direct physical determinants of an atom's ionization energy. The regressions are **exploratory statistical models**, not mechanistic theories.

Q5(e): What would improve the project? Add well-motivated features such as atomic number, periodic-table group/period, valence electronic configuration, or electronegativity, taking care to distinguish features available at prediction time from potentially circular variables. Expand and audit the source dataset, use repeated/nested validation, and assess performance separately across element families. Use sources with clearly documented measurement conditions and uncertainties.

Data limitations: A number of entries use different phases, allotropes, or conventions. Our conservative parser leaves ambiguous entries missing rather than inventing a value. Median imputation is a practical demonstration, not a physically justified replacement for experimental properties. We do not infer accuracy beyond the supplied sample.

Complete student submission checklist

1. Execute every code cell without errors.
2. Explain the five input features, target units, and why this is regression.
3. Show the dataset-cleaning report and justify how ambiguous fields are treated.
4. Reproduce the hand-calculated toy slope, intercept, predictions, and MSE.
5. Report OLS coefficients and explain the effect of standardization.
6. Report the cross-validated Ridge penalty and its validation score.
7. Include the model comparison table, three parity plots, and residual plot.
8. Include and discuss the five largest errors for each model.
9. Write a paragraph about chemical interpretation, leakage, overfitting, and generalization.

Instructor grading guidance

Category	Point allocation
Scientific context, variables, and units	15
Parsing and missing-data handling, separation of train/test	20
Mathematical derivation, hand calculation, OLS implementation	20
Ridge penalty, cross-validation, validation plot	20
Random forest, metrics and plots, outlier table, scientific critique	25
Total	**100**

Reproducibility note: The CSV is incorporated from the original assignment ZIP. The notebook does not require `periodic_table.txt` or the separate general scikit-learn tutorial notebook. The code uses current scikit-learn APIs and avoids hard-coded Colab Drive paths.